

Temario detallado — Programa de formación interna de Stimulo

Mismos módulos y apartados que ya están en el repositorio ([js/content.js](#)), pero con **10 subtemas concretos dentro de cada apartado** para darle profundidad. Cada subtema es un punto que ChatGPT (o quien sea) puede **ampliar con conocimiento** en el deep research. Estructura: 4 secciones · 20 módulos · 5 apartados · 10 subtemas = 1.000 puntos.

Apartados de cada módulo: 📖 Introducción · 🧱 Fundamentos · 💡 Ejemplos prácticos · 🎯 Ejercicios (~2,5 min) · 🔗 Recursos.

Sección 1 · Fundamentos

1. `prompt-engineering` · Prompt Engineering

La base de todo: cómo hablarle a Claude para obtener exactamente lo que necesitas. Etiquetas: Zero-shot, Few-shot, Chain-of-thought, Role prompting.

📖 Introducción

1. Qué es un prompt y por qué el resultado depende de él, no del modelo.
2. La fórmula base: claridad + contexto + formato.
3. Diferencia entre pedir y dirigir (orden suelta vs. instrucción completa).
4. El coste de la ambigüedad: prompts vagos que fallan.
5. Cómo "interpreta" Claude un prompt (predicción guiada por instrucciones).
6. Mitos: "la IA me lee la mente" y por qué no.
7. Cuándo basta un prompt simple y cuándo hace falta estructura.
8. El prompt como activo reutilizable, no algo a improvisar cada vez.
9. Aplicación por área: diseño, marketing, ingeniería, compras/admin.
10. Qué mejora el prompting con los modelos recientes (Opus 4.x).

🧱 Fundamentos

1. Zero-shot: cuándo funciona y sus límites.
2. Few-shot: cómo elegir 2-3 ejemplos representativos.
3. Chain-of-thought: forzar razonamiento paso a paso.
4. Role prompting: asignar rol de experto y su efecto en el output.
5. Especificidad: longitud, tono, formato y audiencia.
6. Delimitadores y estructura (comillas, etiquetas).
7. Instrucciones positivas vs. negativas ("haz" mejor que "no hagas").

8. Orden de la información dentro del prompt.
9. Iteración: refinar el prompt a partir del resultado.
10. Plantillas de prompt para tareas recurrentes.



Ejemplos prácticos

1. Email a cliente: versión vaga vs. específica (caso Müller).
2. Few-shot para clasificar feedback de clientes.
3. Chain-of-thought para decidir entre 3 proveedores.
4. Role prompting: "actúa como director de arte".
5. Resumen de reunión con formato fijo.
6. Reescritura de un texto en 3 tonos distintos.
7. Generación de nombres de campaña con restricciones.
8. Briefing de diseño a partir de notas sueltas.
9. Checklist de QA de un entregable.
10. Prompt que se autocorrige ("revisa tu respuesta antes de darla").



Ejercicios (~2,5 min)

1. Reescribe un prompt malo añadiendo rol + contexto + formato.
2. Convierte una orden en few-shot con 2 ejemplos.
3. Añade "piensa paso a paso" y compara resultados.
4. Pide la misma tarea con 3 roles distintos.
5. Fija un formato de salida (viñetas, máx. palabras).
6. Usa delimitadores para separar instrucción de datos.
7. Pide a Claude que mejore tu propio prompt.
8. Crea una plantilla con variables `{{...}}`.
9. Provoca un fallo con ambigüedad y luego arréglalo.
10. Prompt de auto-revisión sobre un texto propio.



Recursos

1. Guía oficial de prompt engineering (Anthropic).
2. Prompt library de Anthropic.
3. Prompt generator/improver de la consola.
4. Documento de uso de etiquetas XML.
5. Documento de chain-of-thought.
6. Ejemplos del cookbook de Anthropic.
7. Plantilla interna de prompts del estudio.
8. Cheatsheet de patrones (zero/few/CoT/role).
9. Casos de prompts por área del estudio.
10. Checklist "antes de enviar el prompt".

2. gestion-contexto · Gestión del contexto

Cómo Claude usa la memoria de la conversación y cómo estructurarla a tu favor. Etiquetas: Ventana de contexto, System prompt, Historial.

Introducción

1. Qué es la ventana de contexto y por qué todo "vive" ahí.
2. Claude no recuerda entre conversaciones: implicaciones prácticas.
3. La metáfora de la mesa de trabajo: orden = mejor rendimiento.
4. Más contexto no siempre es mejor (el ruido distrae).
5. Coste y tiempo según el tamaño del contexto.
6. Cuándo reiniciar una conversación vs. seguir en ella.
7. El system prompt como reglas permanentes de la sesión.
8. Riesgo de contradicciones acumuladas en el historial.
9. Aplicación: trabajar con documentos largos del estudio.
10. Qué permiten las ventanas grandes de los modelos actuales.

Fundamentos

1. Tokens: qué son y cómo se cuentan.
2. Estructura óptima: instrucción → datos → pregunta.
3. Posición: lo del principio y el final pesa más.
4. Etiquetas para encapsular documentos (`<documento>`).
5. System prompt vs. mensaje de usuario.
6. Gestión del historial largo (resumir, podar).
7. Anclar respuestas a las fuentes del contexto.
8. Evitar fugas y contradicciones de contexto.
9. Prompt caching para contenido repetido.
10. Estrategias cuando la ventana se llena.

Ejemplos prácticos

1. Pegar un PDF con etiquetas `<documento>` .
2. Acta de reunión + pregunta concreta al final.
3. System prompt para un asistente del estudio.
4. Resumen anclado: "solo lo que aparezca en el texto".
5. Reiniciar la conversación con contexto limpio.
6. Comparar dos versiones de un contrato.
7. Pliego técnico largo → extracción de requisitos.
8. Mantener el tono de marca vía system prompt.
9. Hilo de emails → resumen accionable.
10. Base de conocimiento del estudio cargada en contexto.

Ejercicios (~2,5 min)

1. Pega un documento con la instrucción antes y la pregunta después.
2. Escribe un system prompt para tu rol.
3. Compara la respuesta con y sin etiquetas.
4. Pide "di 'no aparece' si falta el dato".
5. Resume un hilo y límtalo a 5 puntos.
6. Reinicia con solo el contexto bueno.
7. Estima los tokens de un texto.
8. Estructura un prompt en 3 bloques.
9. Detecta una contradicción introducida a propósito.
10. Crea un system prompt reutilizable de equipo.

Recursos

1. Long context tips (Anthropic).
2. Uso de etiquetas XML.
3. Documento de system prompts.
4. Documento de prompt caching.
5. Tokenizer / conteo de tokens.
6. Guía de trabajo con PDFs.
7. Plantilla de system prompt del estudio.
8. Ejemplos de RAG ligero.
9. Buenas prácticas de ventana de contexto.
10. Checklist de estructura de prompt.

3. **razonamiento** · Razonamiento y análisis

Cómo piensa Claude y cómo activar su capacidad analítica más profunda. Etiquetas: Think step by step, Extended thinking, Razonamiento crítico.

Introducción

1. Por qué una respuesta inmediata es intuición, no análisis.
2. Dar "espacio para pensar" mejora la precisión.
3. Tareas que se benefician: lógica, cálculo, decisiones.
4. Qué es extended thinking y cuándo activarlo.
5. Coste y latencia del razonamiento extendido.
6. Razonamiento visible vs. oculto.
7. La autocrítica como red de seguridad.
8. Mitos: "la IA siempre acierta los números".
9. Aplicación: compras, presupuestos, planificación.
10. Mejoras de razonamiento en los modelos 4.x.

Fundamentos

1. Chain-of-thought explícito.
2. Extended thinking (modo de razonamiento).
3. Separar `<análisis>` de `<respuesta>`.
4. Descomposición de problemas en pasos.
5. Verificación y autocrítica.
6. Razonamiento contrafactual ("¿y si...?").
7. Detección de supuestos ocultos.
8. Comprobación numérica paso a paso.
9. Cuándo NO sobre-razonar (tareas simples).
10. Comunicar la incertidumbre.

Ejemplos prácticos

1. Elegir proveedor con 3 criterios ponderados.
2. Calcular el coste total con transporte.
3. Planificar las fases de un proyecto.
4. Detectar un error en un presupuesto.
5. Pros y contras de dos enfoques de diseño.
6. Análisis de riesgo de plazo.
7. Revisión escéptica de una recomendación.
8. Estimación con rango (mejor/peor caso).
9. Diagnóstico de por qué falló una campaña.
10. Decisión make-or-buy.

Ejercicios (~2,5 min)

1. Pide razonar paso a paso una decisión real.
2. Pide que critique su propia respuesta.
3. Separa análisis y conclusión con etiquetas.
4. Da un problema numérico y verifica el resultado.
5. Pide 3 supuestos que podrían estar mal.
6. Compara respuesta directa vs. razonada.
7. Pide un rango en vez de un número único.
8. Activa extended thinking en un caso difícil.
9. Pregunta "¿qué dato te haría cambiar de opinión?".
10. Resuelve un reto de lógica corto.

Recursos

1. Extended thinking (documento).
2. Chain-of-thought (documento).
3. Cookbook de razonamiento.

4. Guía de evaluación de respuestas.
5. Plantilla análisis → decisión.
6. Casos de decisión del estudio.
7. Documento sobre incertidumbre/calibración.
8. Ejercicios de pensamiento crítico.
9. Checklist de verificación numérica.
10. Comparativa de modelos para razonamiento.

4. **creatividad** • Creatividad y escritura

Claude como co-creador: voz, tono, estilo y escritura estratégica. Etiquetas: Tono y voz, Iteración creativa, Copywriting, Storytelling.

Introducción

1. Claude escribe bien por defecto, pero genérico.
2. Definir la voz con ejemplos, no con adjetivos.
3. Audiencia + objetivo lo cambian todo.
4. La escritura como proceso iterativo.
5. El problema del texto que "suena a IA".
6. Creatividad dirigida vs. azar.
7. Co-creación: tú diriges, Claude propone volumen.
8. Aplicación: marketing, propuestas, redes, branding.
9. Mitos: "la IA mata la creatividad".
10. Qué aporta el estudio (criterio) y qué la IA (cantidad y velocidad).

Fundamentos

1. Perfil de voz (5 rasgos) reutilizable.
2. Clonar estilo a partir de muestras propias.
3. Briefing creativo: audiencia, objetivo, tono.
4. Iterar por capas (estructura → contenido → pulido).
5. Pedir variantes y ángulos distintos.
6. Frameworks de copywriting (AIDA, PAS).
7. Storytelling: estructura narrativa básica.
8. Editar las "frases que suenan a IA".
9. Restricciones que potencian la creatividad.
10. Consistencia de marca a escala.

Ejemplos prácticos

1. Post de LinkedIn con la voz del estudio.
2. Naming de producto con restricciones.
3. Tres ganchos de apertura (dolor/aspiración/dato).

4. Reescribir un texto en formal/cercano/provocador.
5. Guion de vídeo de 30 segundos.
6. Email frío de captación.
7. Descripción de producto/servicio.
8. Caso de éxito a partir de datos sueltos.
9. Newsletter mensual del estudio.
10. Microcopy para una interfaz.

Ejercicios (~2,5 min)

1. Pega 2 textos tuyos y pide tus 5 rasgos de voz.
2. Pide un texto nuevo imitando esa voz.
3. Pide 3 ángulos de gancho para una idea.
4. La misma idea en 3 tonos.
5. Marca las frases que no suenan a ti y reescríbelas.
6. Aplica el framework AIDA a un servicio.
7. Escribe microcopy para un botón/aviso.
8. Naming con 3 restricciones.
9. Reescribe un párrafo aburrido para que enganche.
10. Guion de 30s para redes.

Recursos

1. Prompt library (marketing y escritura).
2. Guía de tono y estilo.
3. Ejemplos de copy de alto rendimiento.
4. Frameworks de copywriting (AIDA/PAS).
5. Plantilla de briefing creativo.
6. Banco de voz del estudio (5 rasgos guardados).
7. Documento de iteración creativa.
8. Casos de campañas reales.
9. Checklist anti-"suena a IA".
10. Herramientas de redacción y revisión.

5. · Código y tareas técnicas

Claude como engineer: debug, generación, revisión y arquitectura. Etiquetas: Debug, Code review, Arquitectura, Explicación técnica.

Introducción

1. Claude como par de programación.
2. No hace falta ser programador para aprovecharlo.
3. Dar contexto: error + código + resultado esperado.

4. Los cuatro trabajos: generar, depurar, revisar, explicar.
5. Lenguajes y herramientas del estudio (sobre todo Python).
6. Mitos: "el código de IA no es fiable".
7. La revisión humana es imprescindible.
8. De la idea a un script funcional.
9. Aplicación: automatizar tareas internas.
10. Mejoras de coding en los modelos 4.x y en Claude Code.



Fundamentos

1. Especificar lenguaje, versión y librerías.
2. Pegar el stack trace completo.
3. Pedir tests junto al código.
4. Code review ordenado por gravedad.
5. Explicación adaptada al nivel.
6. Refactor y legibilidad.
7. Manejo de errores y casos límite.
8. Arquitectura de un script pequeño.
9. Iterar ejecutando de verdad.
10. Seguridad básica: no exponer secretos.



Ejemplos prácticos

1. Debug de una función que devuelve None.
2. Script de renombrado masivo de archivos.
3. Review de un endpoint.
4. Explicar una librería desconocida.
5. Convertir una macro VBA a Python.
6. Tests para una función de cálculo de IVA.
7. Refactor de código duplicado.
8. Leer un CSV y resumirlo.
9. Script de copia de seguridad.
10. Documentar una función con docstrings.



Ejercicios (~2,5 min)

1. Pega un error y pide causa + fix mínimo.
2. Pide una review por gravedad.
3. Pide un test que demuestre el bug.
4. Genera una función con type hints.
5. Pide que te expliquen tu propio código.
6. Refactoriza 10 líneas.
7. Añade manejo de errores a una función.

8. Convierte pseudocódigo a Python.
9. Documenta con docstrings.
10. Pide alternativas de implementación.

Recursos

1. Claude Code (documentación).
2. Cookbook de Anthropic.
3. Documento de tool use.
4. Guía de buenas prácticas de coding.
5. Documentación de Python.
6. Plantilla de prompt de debug.
7. Repos de ejemplo.
8. Checklist de code review.
9. Documento de testing.
10. Guía de gestión de secretos.

6. **investigacion** · Investigación y análisis

Claude como analista: síntesis, comparación de fuentes e insights estratégicos. Etiquetas: Síntesis de docs, Comparación, Extracción de insights.

Introducción

1. Cuando hay más información de la que puedes leer.
2. Claude como analista, no como buscador.
3. Una buena pregunta da un buen análisis.
4. Exigir trazabilidad a las fuentes.
5. De "léete esto" a "qué decido y por qué".
6. Riesgo de invención sin anclaje.
7. Diferencia entre síntesis, resumen e insight.
8. Aplicación: estudios de mercado, benchmarking, pliegos.
9. Mitos: "la IA cita siempre bien".
10. Ventana grande para procesar muchos documentos.

Fundamentos

1. Síntesis con cita de fuente.
2. Tablas comparativas con criterios.
3. Extracción de insights no obvios.
4. Gestión de la incertidumbre con (?).
5. Detección de contradicciones.
6. De análisis a recomendación.
7. Triangulación de fuentes.

8. Sesgos y cómo mitigarlos.
9. Estructurar la salida (tabla/JSON).
10. Verificación de los datos clave.



Ejemplos prácticos

1. Comparar 3 informes de proveedor.
2. 20 respuestas de clientes → 3 temas.
3. Benchmark de competencia.
4. Resumen de un pliego técnico.
5. Comparar dos contratos.
6. Extraer requisitos de un brief.
7. Análisis DAFO asistido.
8. Insights a partir de datos de ventas.
9. Mini revisión de literatura.
10. Detectar el problema que nadie menciona.



Ejercicios (~2,5 min)

1. Tabla comparativa de 3 fuentes con citas.
2. Pide recomendación + "qué la cambiaría".
3. Marca con (?) lo no respaldado.
4. Extrae 3 temas recurrentes.
5. Detecta contradicciones entre fuentes.
6. Resume anclado a la fuente.
7. Saca un insight contraintuitivo.
8. Convierte el análisis en una decisión.
9. Estructura la salida en tabla.
10. Verifica un dato clave.



Recursos

1. Guía de summarization.
 2. Long context tips.
 3. Documento de etiquetas XML.
 4. Cookbook de análisis.
 5. Plantilla de tabla comparativa.
 6. Casos de benchmarking.
 7. Guía de citación/anclaje.
 8. Herramientas de extracción.
 9. Checklist anti-alucinación.
 10. Documento de structured output.
-

Sección 2 · Técnicas avanzadas

7. meta-prompting · Estructuras y meta-prompting

System prompts, roles de experto, XML y plantillas reutilizables. Etiquetas: Meta-prompting, XML estructurado, Plantillas, Roles de experto.

Introducción

1. De improvisar prompts a tener activos reutilizables.
2. Plantillas para tareas que se repiten.
3. Meta-prompting: que Claude escriba el prompt por ti.
4. Consistencia del output como objetivo.
5. El system prompt como contrato.
6. Cuándo merece la pena crear una plantilla.
7. De usuario a "ingeniero de prompts".
8. Aplicación: procesos repetibles del estudio.
9. Mitos: "cada prompt desde cero".
10. Plantillas + JSON para automatizar después.

Fundamentos

1. System prompt: rol + reglas + formato.
2. XML para estructurar (`<contexto>` , `<task>`).
3. Variables `{{...}}` en plantillas.
4. Meta-prompting paso a paso.
5. Salida en JSON con esquema.
6. Roles de experto compuestos.
7. Versionar las plantillas.
8. Validar la salida estructurada.
9. Librería de plantillas del equipo.
10. Pruebas A/B de prompts.

Ejemplos prácticos

1. Plantilla XML de revisión de no conformidad.
2. Meta-prompt para clasificar emails.
3. Plantilla de propuesta comercial.
4. Salida JSON para integrar con otro sistema.
5. Rol "auditor de calidad de fabricación".
6. Plantilla de brief de diseño.
7. Plantilla de resumen de reunión.
8. Generador de prompts del estudio.

9. Plantilla de presupuesto.
10. Plantilla de respuesta a cliente.

Ejercicios (~2,5 min)

1. Crea una plantilla con `{{variables}}`.
2. Pide a Claude que escriba el prompt (meta-prompting).
3. Pide una salida JSON válida.
4. Define un system prompt de rol.
5. Estructura un prompt con etiquetas XML.
6. Versiona dos variantes y compáralas.
7. Valida el JSON generado.
8. Convierte un prompt suelto en plantilla.
9. Crea un rol experto compuesto.
10. Guárdalo en la librería de equipo.

Recursos

1. Documento de system prompts.
2. Increase consistency / JSON (documento).
3. Documento de etiquetas XML.
4. Prompt generator de la consola.
5. Cookbook de structured output.
6. Plantillas del estudio.
7. Guía de meta-prompting.
8. Documento de tool use (JSON → acción).
9. Repositorio de plantillas.
10. Checklist de validación.

8. `workflows` · Workflows y automatización

Claude en flujos encadenados, API y sistemas agénticos. Etiquetas: Flujos en cadena, API, Agentes IA, Automatización.

Introducción

1. De chatear a poner a Claude a trabajar solo.
2. Claude como componente, no como ventana de chat.
3. Qué es el prompt chaining.
4. Qué es la API.
5. Qué es un agente.
6. Cuándo automatizar y cuándo no.
7. Riesgos de un agente sin frenos.
8. Aplicación: automatizaciones del estudio.

9. Mitos: "automatizar es muy difícil".
10. Estado de los sistemas agénticos en 2025-2026.

Fundamentos

1. Prompt chaining (dividir la tarea).
2. Llamada básica a la API.
3. Tool use / function calling.
4. Bucle agéntico: plan → ejecutar → observar.
5. Límites de iteraciones y validación.
6. Idempotencia y errores.
7. Observabilidad y logging.
8. Orquestación de varios pasos.
9. Coste y latencia.
10. Seguridad y humano en el bucle.

Ejemplos prácticos

1. Flujo email → pedido → confirmación.
2. Llamada a la API en Python.
3. Cadena extraer → validar factura.
4. Tool use con una función propia.
5. Clasificar y enrutar tickets.
6. Generar y publicar un borrador (con revisión).
7. Pipeline de informe semanal.
8. Webhook → Claude → Slack.
9. Resumen diario automático.
10. Agente que corrige su propio error.

Ejercicios (~2,5 min)

1. Diseña un flujo de 3 pasos en papel.
2. Define entradas y salidas de cada paso.
3. Haz una llamada básica a la API.
4. Define una herramienta (tool).
5. Marca dónde interviene el humano.
6. Pon un tope de iteraciones.
7. Encadena dos prompts.
8. Loguea cada paso.
9. Maneja un error simulado.
10. Estima el coste del flujo.

Recursos

1. Tool use (documento).

2. Building effective agents (Anthropic).
 3. SDK de Python.
 4. Cookbook de agents.
 5. Documento de la API Messages.
 6. MCP (documento).
 7. Ejemplos de chaining.
 8. Guía de error handling.
 9. Plantilla de diseño de flujo.
 10. Checklist de seguridad de agentes.
-

Sección 3 · Ingeniería mecánica y diseño 3D

9. `blender` · Claude + Blender

Agente que opera Blender vía `bpy` : geometría paramétrica, materiales y render. Etiquetas: bpy API, Geometría paramétrica, Render batch, Scripting agéntico.

Introducción

1. Blender trae un intérprete de Python (`bpy`) completo.
2. El puente: describir → Claude genera `bpy` → Blender ejecuta.
3. Casos: paramétrico, render, procesos por lotes.
4. Quién se beneficia (diseño 3D y de producto).
5. La versión de Blender importa (3.x vs 4.x).
6. Mitos: "hay que ser programador".
7. Del clic al script.
8. Aplicación: variantes de producto y visuales.
9. Riesgos: scripts que modifican la escena.
10. Estado de `bpy` en Blender 4.x.

Fundamentos

1. `bpy.ops` vs `bpy.data` .
2. Ejecutar scripts (pestaña Scripting / por CLI).
3. Parametrizar con variables al inicio.
4. Sistema de coordenadas y unidades.
5. Materiales y nodos por código.
6. Cámaras, luces y render.
7. Render por lotes de variantes.
8. Modificadores por script.
9. Exportar (STL/OBJ/FBX).

10. Depurar errores de `bpy` .



Ejemplos prácticos

1. Cubo paramétrico.
2. Soporte en L con fillet.
3. Generar 3 variantes en un bucle.
4. Asignar un material por código.
5. Render por lotes a PNG.
6. Array de piezas.
7. Importar y posicionar un mesh.
8. Cámara automática apuntando a un objeto.
9. Exportar a STL.
10. Escena limpia con setup de luz.



Ejercicios (~2,5 min)

1. Script de primitiva con 2-3 parámetros.
2. Cambia un parámetro y reejecuta.
3. Bucle de 3 variantes.
4. Asigna un material.
5. Configura una cámara.
6. Renderiza a un archivo.
7. Aplica un modificador.
8. Exporta a STL.
9. Activa Python tooltips y copia un comando.
10. Pide a Claude el script para un objeto descrito.



Recursos

1. Blender Python API (`bpy`).
2. Tutoriales de scripting en Blender.
3. Ejemplos/cookbook de `bpy` .
4. Documento de render por CLI.
5. Foro/Stack de Blender.
6. Plantilla de script paramétrico.
7. Guía de materiales por nodos.
8. Documento de exportadores.
9. Repos de add-ons de ejemplo.
10. Checklist de versión de la API.

10. `solidworks` · Claude + SolidWorks

Agente que controla SolidWorks vía API COM: diseño paramétrico y planos. Etiquetas: win32com, Diseño paramétrico, Configuraciones, Export DXF/STEP.

Introducción

1. SolidWorks expone una API COM muy completa.
2. Python la controla con `pywin32`.
3. Casos: cambiar cotas, configuraciones, planos.
4. Quién: ingeniería mecánica.
5. Solo Windows.
6. Mitos: "la API es inaccesible".
7. Grabar una macro y traducirla.
8. Aplicación: familias de piezas.
9. Riesgos: modificar modelos en producción.
10. Estado de la API de SolidWorks.

Fundamentos

1. `Dispatch("SldWorks.Application")`.
2. Jerarquía App → Document → Feature.
3. Cotas con nombre (`D1@Croquis1`).
4. `EditRebuild3` para reconstruir.
5. Configuraciones.
6. Export STEP/IGES/DXF/PDF.
7. Propiedades personalizadas.
8. Recorrer features.
9. Manejo de errores COM.
10. Automatizar planos.

Ejemplos prácticos

1. Conectar y cambiar una cota.
2. Bucle de anchos → variantes.
3. Exportar a STEP.
4. Crear una configuración.
5. Leer propiedades del modelo.
6. Exportar un plano a PDF.
7. DXF de chapa.
8. Cambiar el material.
9. Recorrer un ensamblaje.
10. Generar un BOM básico.

Ejercicios (~2,5 min)

1. Anota el nombre de una cota.
2. Script que la cambie y reconstruya.
3. Exporta a STEP.
4. Bucle de 3 medidas.
5. Lee una propiedad.
6. Exporta un plano a PDF.
7. Graba una macro y pide su traducción.
8. Crea una configuración.
9. Maneja un error COM.
10. Repasa el script línea a línea.

Recursos

1. SolidWorks API Help.
2. `pywin32`.
3. Foros de la API de SW.
4. Ejemplos VBA → Python.
5. Documento de export.
6. Plantilla de script paramétrico.
7. Guía de configuraciones.
8. Documento de propiedades.
9. Repos de macros.
10. Checklist de seguridad de modelos.

11. `fusion360` · Claude + Fusion 360

Agente que diseña en Fusion 360 vía su API de Python. Etiquetas: Fusion API, Sketches auto, Variantes paramétricas, Export STL/STEP.

Introducción

1. Fusion 360 tiene una API de Python nativa.
2. Editor de scripts integrado.
3. El CAD más cómodo para empezar.
4. Scripts vs. add-ins.
5. Multiplataforma.
6. Mitos sobre la dificultad.
7. De un parámetro a un sólido.
8. Aplicación: variantes paramétricas.
9. Riesgos al automatizar.
10. Estado de la API de Fusion.

Fundamentos

1. `adsk.core` / `adsk.fusion`.
2. `run(context)` como punto de entrada.
3. Sketches y perfiles.
4. Extrusiones y operaciones.
5. User Parameters.
6. Timeline y features.
7. `ExportManager` (STL/STEP).
8. Componentes y ocurrencias.
9. Manejo de errores con `traceback`.
10. Add-ins con interfaz propia.

Ejemplos prácticos

1. Esqueleto de un script.
2. Círculo + extrusión.
3. R y H como User Parameters.
4. Caja paramétrica.
5. Patrón de agujeros.
6. Export a STL.
7. Export a STEP.
8. Variante por parámetros.
9. Sketch desde puntos.
10. Add-in mínimo.

Ejercicios (~2,5 min)

1. Crea un script nuevo en Fusion.
2. Dibuja un círculo y extrúyelo.
3. Define User Parameters.
4. Cambia un parámetro y reejecuta.
5. Exporta a STL.
6. Crea un patrón rectangular.
7. Maneja un error con `traceback`.
8. Abre un ejemplo oficial.
9. Exporta a STEP.
10. Pide a Claude un script para una pieza descrita.

Recursos

1. Fusion 360 API & Scripts.
2. Ejemplos oficiales en el editor.
3. Foros de Autodesk.

4. Documento de `ExportManager`.
5. Guía de User Parameters.
6. Plantilla de script.
7. Tutoriales de add-ins.
8. Repos de ejemplo.
9. Documento de sketches.
10. Checklist de la API.

12. agente-python · Agente Python agéntico con Claude API

El bucle completo: Claude planifica, Python ejecuta, el resultado vuelve a Claude. Etiquetas: anthropic SDK, Tool use, Bucle plan→ejecutar→corregir, Error handling.

Introducción

1. El corazón de toda la sección de ingeniería.
2. De copiar scripts a un bucle automático.
3. Claude decide, Python ejecuta, el error vuelve a Claude.
4. Reúne API + tool use + manejo de errores.
5. Reutilizable para Blender / SolidWorks / Fusion.
6. Mitos: "los agentes son magia".
7. Frenos de seguridad imprescindibles.
8. Aplicación: agente de ingeniería.
9. Coste y latencia.
10. Estado del tool use en 2025-2026.

Fundamentos

1. Definir herramientas (con su esquema).
2. El bucle de `messages`.
3. `stop_reason == tool_use`.
4. Devolver `tool_result`.
5. Tope de iteraciones.
6. Sandbox para ejecutar código.
7. Validación de inputs.
8. Manejo y reintento de errores.
9. Logging y observabilidad.
10. Humano en el bucle.

Ejemplos prácticos

1. Esqueleto del bucle agéntico.
2. Herramienta `ejecutar_python`.
3. Devolver stdout/errores como resultado.

4. Corregir tras un fallo.
5. Herramienta de leer/escribir archivos.
6. Agente que crea un cubo en Blender.
7. Agente multi-herramienta.
8. Reintento con backoff.
9. Límite de pasos.
10. Log de las decisiones del agente.

Ejercicios (~2,5 min)

1. Define una herramienta sobre papel.
2. Escribe el objetivo en lenguaje natural.
3. Describe cada vuelta del bucle.
4. Completa el esqueleto con Claude.
5. Añade un tope de iteraciones.
6. Maneja un error y reintent.
7. Loguea cada paso.
8. Marca dónde va el humano.
9. Valida un input.
10. Prueba con una tarea simple.

Recursos

1. Tool use (documento).
2. Building effective agents.
3. Cookbook de agents.
4. SDK de Python.
5. Documento de la Messages API.
6. MCP.
7. Ejemplos de tool use.
8. Guía de error handling.
9. Plantilla de bucle agéntico.
10. Checklist de seguridad.

13. **casos-ingeniería** · Casos de uso para ingeniería

10 automatizaciones reales: variantes, BOM, tolerancias, documentación. Etiquetas: Variantes de pieza, BOM automático, Análisis tolerancias, Documentación auto.

Introducción

1. Ya tienes las piezas (CAD + Python + agente).
2. Qué construir primero.
3. Medir el ahorro de tiempo.

4. Empezar por lo simple.
5. ROI por caso.
6. Mitos: "hay que automatizar todo de golpe".
7. La prueba de concepto.
8. Aplicación transversal.
9. Riesgos de automatizar mal.
10. Escalar con datos que lo demuestren.

Fundamentos (los 10 casos)

1. Variantes de pieza.
2. BOM automático.
3. Renombrado y normalización.
4. Análisis de tolerancias.
5. Documentación / hoja de pieza.
6. Comparar revisiones.
7. Export multiformato.
8. Cálculos de diseño (peso, coste).
9. Generación de planos.
10. Informe de proyecto.

Ejemplos prácticos

1. Tabla de medidas → N modelos.
2. BOM → CSV agrupado.
3. Normalizar nombres de archivos.
4. Check de cotas dentro de rango.
5. Ficha técnica automática.
6. Diff entre v1 y v2.
7. STEP + STL + DXF + PDF de un golpe.
8. Peso y coste de material.
9. Vistas y cotas básicas.
10. PDF de proyecto consolidado.

Ejercicios (~2,5 min)

1. Elige el caso con más ahorro esta semana.
2. Define input / proceso / output en 3 líneas.
3. Pide a Claude el primer paso.
4. Estima minutos/semana ahorrados.
5. Ordena los 10 casos por ROI.
6. Esboza el flujo BOM → CSV.
7. Diseña la ficha de pieza.

8. Define reglas de tolerancia.
9. Lista los formatos de export.
10. Define el informe final.

Recursos

1. Use case guides (Anthropic).
2. Documento de export por CAD.
3. Ejemplos de BOM.
4. Plantilla de ficha de pieza.
5. Guía de tolerancias.
6. Cookbook.
7. Plantilla de informe.
8. Casos reales del sector.
9. Checklist de ROI.
10. Documento de cálculo de diseño.

Sección 4 · Administración, compras y contabilidad con IA

14. **compras** · Automatización de compras

De la BOM de ingeniería a la orden de compra. Etiquetas: BOM → orden de compra, Comparación proveedores, Aprobaciones auto, Python + email/Slack.

Introducción

1. Primer eslabón donde la ingeniería se vuelve dinero.
2. La BOM se transforma en órdenes de compra.
3. Claude agrupa y compara proveedores.
4. Quién: compras / administración.
5. Genera borradores, no envíos automáticos.
6. Mitos sobre la automatización.
7. Del CAD al aprovisionamiento.
8. Aplicación directa.
9. Riesgos: enviar sin revisar.
10. Estado de las integraciones.

Fundamentos

1. La BOM como entrada (CSV).
2. Agrupación y consolidación.

3. Comparación de proveedores.
4. Reglas de aprobación.
5. Salida accionable (borrador).
6. Integración con email/Slack.
7. Histórico de fiabilidad.
8. Mínimos de pedido.
9. Validación de datos.
10. Humano en el gatillo.



Ejemplos prácticos

1. BOM + tarifas → proveedor óptimo por línea.
2. Borrador de orden por proveedor.
3. Marcar pedidos >1.000€ para aprobación.
4. Email de pedido.
5. Comparar 3 proveedores.
6. Detectar stock ya disponible.
7. Consolidar piezas comunes.
8. Aviso por Slack.
9. Calcular el coste total.
10. Resumen de compras del mes.



Ejercicios (~2,5 min)

1. BOM de 5 líneas → óptimo por línea.
2. Genera el borrador de pedido.
3. Aplica la regla ">500€ a revisión".
4. Revisa la lógica de elección.
5. Compara 2 proveedores.
6. Genera el email de pedido.
7. Consolida piezas iguales.
8. Calcula el coste total.
9. Diseña el flujo de aprobación.
10. Marca dónde interviene el humano.



Recursos

1. Tool use (email/Slack).
2. APIs de email/Slack.
3. Plantilla de orden de compra.
4. Guía de comparación de proveedores.
5. Ejemplos de pandas.
6. Checklist de aprobaciones.

7. Documento de seguridad de envíos.
8. Plantilla de BOM.
9. Casos de compras.
10. Cookbook.

15. · Integración con CRM

Claude conecta el proyecto técnico con el cliente. Etiquetas: HubSpot / Salesforce API, Propuestas auto, Seguimiento inteligente, Datos técnicos → oferta.

Introducción

1. El CRM es donde vive la relación con el cliente.
2. Datos técnicos → propuesta comercial.
3. Mantener las oportunidades al día.
4. Seguimiento en el momento oportuno.
5. Quién: comercial / administración.
6. Privacidad de los datos de cliente.
7. Del taller a la oferta.
8. Aplicación directa.
9. Riesgos: exponer datos sensibles.
10. Estado de las APIs de CRM.

Fundamentos

1. API del CRM (REST).
2. Leer/escribir contactos y deals.
3. Datos técnicos → oferta.
4. Seguimiento inteligente.
5. Resumen de cuenta.
6. Plantillas de propuesta.
7. Scoring y priorización.
8. Notas y actividades.
9. Privacidad y permisos.
10. Sincronización.

Ejemplos prácticos

1. Coste técnico → propuesta.
2. Briefing de reunión en 5 puntos.
3. Detectar una oportunidad fría.
4. Crear un deal desde un email.
5. Redactar un seguimiento.
6. Resumen del histórico de cuenta.

7. Priorizar 3 oportunidades.
8. Plantilla de oferta.
9. Email de propuesta.
10. Informe de pipeline.

Ejercicios (~2,5 min)

1. Proyecto + costes → propuesta.
2. Genera un briefing de reunión.
3. Prioriza 3 oportunidades.
4. Redacta un seguimiento.
5. Resume un histórico.
6. Crea un deal desde un email (caso diseño).
7. Crea la plantilla de oferta de empresa.
8. Detecta una oportunidad estancada.
9. Calcula el margen de una oferta.
10. Define qué datos hay que proteger.

Recursos

1. HubSpot API.
2. Salesforce REST API.
3. Plantilla de propuesta.
4. Guía de seguimiento.
5. Documento de tool use.
6. Ejemplos de integración.
7. Checklist de privacidad.
8. Plantilla de briefing.
9. Casos de CRM.
10. Cookbook.

16. contabilidad · Fundamentos de contabilidad + IA

Lo que necesitas saber de contabilidad para automatizarla. Etiquetas: Asientos contables, IVA automatizado, Cierre mensual, Clasificación de gastos.

Introducción

1. No hace falta ser contable para automatizar.
2. Sí hace falta entender el lenguaje.
3. La IA acelera; la responsabilidad sigue siendo humana.
4. Quién: administración.
5. No es asesoramiento fiscal.
6. Mitos sobre la contabilidad.

7. Lo mínimo imprescindible.
8. Aplicación directa.
9. Riesgos legales y fiscales.
10. Validación profesional siempre.

Fundamentos

1. Partida doble (Debe = Haber).
2. Plan contable y cuentas.
3. IVA repercutido y soportado.
4. Tipos de IVA (21/10/4).
5. Cierre mensual.
6. Conciliación bancaria.
7. Clasificación de gastos.
8. Asientos tipo.
9. Modelos/declaraciones (visión general).
10. Trazabilidad y auditoría.

Ejemplos prácticos

1. Asiento de un gasto con IVA.
2. IVA a liquidar del trimestre.
3. Clasificar 3 gastos.
4. Asiento de una venta.
5. Conciliar extracto vs. factura.
6. Cierre mensual paso a paso.
7. Explicar un concepto con ejemplo.
8. Detectar un gasto mal clasificado.
9. Cuadrar Debe/Haber.
10. Resumen contable del mes.

Ejercicios (~2,5 min)

1. 3 gastos → sus asientos.
2. Verifica que Debe = Haber.
3. Calcula el IVA a liquidar.
4. Clasifica un gasto dudoso.
5. Asiento de una venta.
6. Pide que te expliquen el IVA con un ejemplo.
7. Concilia 2 movimientos.
8. Detecta un error introducido.
9. Resume el periodo.
10. Marca qué debe validar el asesor.

Recursos

1. Agencia Tributaria (IVA).
2. Plan General Contable (BOE).
3. Guía de cierre.
4. Plantilla de asientos.
5. Documento de clasificación de gastos.
6. Ejemplos de conciliación.
7. Checklist de cierre.
8. Glosario contable.
9. Casos del estudio.
10. Aviso: validación profesional.

17. facturas · Procesador automático de facturas

OCR + Claude extrae, interpreta y clasifica facturas; valida contra OC. Etiquetas: OCR + PyMuPDF, Claude clasifica, Validación vs OC, Export a contabilidad.

Introducción

1. Tarea tediosa y propensa a errores.
2. OCR + Claude extrae, clasifica y valida.
3. ROI rápido y muy visible.
4. Quién: administración / compras.
5. PDF nativo vs. escaneo.
6. Mitos sobre el OCR.
7. Del papel al asiento.
8. Aplicación directa.
9. Riesgos: datos mal extraídos.
10. Estado de la visión/OCR.

Fundamentos

1. Extracción: PyMuPDF / pdfplumber.
2. OCR (Tesseract) o visión.
3. Claude → JSON estructurado.
4. Validación contra la OC.
5. Clasificación contable.
6. Export (asiento / CSV).
7. Campos `null` si faltan.
8. Manejo de formatos variados.
9. Control de errores.
10. Trazabilidad.

Ejemplos prácticos

1. Factura → JSON.
2. Validar base + IVA = total.
3. Cruzar con la OC.
4. Detectar una diferencia de precio.
5. Clasificar a una cuenta.
6. Export a CSV contable.
7. Factura escaneada con OCR.
8. Lote de facturas.
9. Marcar campos faltantes.
10. Aviso de discrepancia.

Ejercicios (~2,5 min)

1. Factura → JSON estructurado.
2. Verifica base + IVA = total.
3. Cruza con una OC ficticia.
4. Introduce y detecta una diferencia.
5. Clasifica a una cuenta.
6. Pide `null` en los campos ausentes.
7. Export a CSV.
8. Procesa 2 facturas.
9. Maneja un formato raro.
10. Define la validación.

Recursos

1. PyMuPDF.
2. Vision (documento).
3. pdfplumber.
4. Tesseract OCR.
5. Plantilla de JSON de factura.
6. Guía de validación vs. OC.
7. Cookbook de extracción.
8. Documento de export contable.
9. Checklist anti-inventor.
10. Casos reales.

18. `dashboard` · Dashboard de administración con IA

Claude como analista de negocio: consolida, alerta y explica. Etiquetas: Consolidación de datos, Alertas automáticas, Análisis de margen, Dashboard Python.

Introducción

1. Ver todos los datos juntos.
2. Que alguien te avise de lo importante.
3. Claude como analista de negocio.
4. Quién: dirección / administración.
5. De los datos a las decisiones.
6. Mitos sobre los dashboards.
7. Consolidar las 4 áreas.
8. Aplicación directa.
9. Riesgos: decidir con datos malos.
10. Estado de las herramientas.

Fundamentos

1. Consolidación con pandas.
2. KPIs (margen, gasto vs. presupuesto, cobro).
3. Alertas por umbral.
4. Narrativa automática.
5. Visualización (Streamlit/Plotly).
6. Separar calcular de interpretar.
7. Frescura de los datos.
8. Drill-down.
9. Exportar informes.
10. Automatizar el refresco.

Ejemplos prácticos

1. Margen por proyecto.
2. Resumen ejecutivo en 4 líneas.
3. Alerta de margen $< 10\%$.
4. Gasto vs. presupuesto.
5. Días de cobro.
6. Top 2 peores proyectos.
7. Tendencia mensual.
8. Consolidar compras + ventas.
9. Panel en Streamlit.
10. Informe a dirección.

Ejercicios (~2,5 min)

1. 3-4 proyectos → margen de cada uno.
2. Ordénalos por margen.
3. Resumen ejecutivo en 4 líneas.

4. Define un umbral de alerta.
5. Separa calcular de interpretar.
6. Define el KPI de cobro.
7. Detecta el peor proyecto.
8. Esboza el panel.
9. Define las alertas.
10. Exporta el informe.



Recursos

1. Streamlit.
2. pandas.
3. Plotly.
4. Plantilla de dashboard.
5. Guía de KPIs.
6. Cookbook.
7. Ejemplos de consolidación.
8. Checklist de calidad de datos.
9. Documento de alertas.
10. Casos de negocio.

19. **herramientas-propias** · Crear herramientas propias de gestión

Construye tus apps internas con Claude como motor. Etiquetas: Streamlit / Flask, Presupuestos técnicos, Coste de proyecto, Claude como motor.



Introducción

1. Dejar las herramientas genéricas y construir las tuyas.
2. Claude genera el código y es el motor inteligente.
3. De semanas a horas.
4. Pensar en producto.
5. Quién: todo el estudio.
6. Mitos sobre programar apps.
7. Un v1 feo pero útil.
8. Aplicación directa.
9. Riesgos: sobre-ingeniería.
10. Estado de Claude Code.



Fundamentos

1. Streamlit para empezar.
2. Flask/FastAPI para crecer.
3. Claude como motor (API).

4. Estado y datos (CSV/SQLite).
5. Iterar con Claude Code.
6. UI mínima.
7. Despliegue (Vercel/Streamlit Cloud).
8. Autenticación básica.
9. Mantenimiento.
10. Cuándo NO construir.



Ejemplos prácticos

1. Calculadora de presupuesto.
2. Gestor de órdenes de trabajo.
3. Calculadora de coste de proyecto.
4. Generador de propuestas.
5. Clasificador de gastos.
6. Buscador interno.
7. Conversor de unidades.
8. App de checklist.
9. Mini-CRM.
10. Panel de KPIs.



Ejercicios (~2,5 min)

1. Identifica un proceso que haces en Excel.
2. Pide el v1 en Streamlit.
3. Localiza dónde llamar a Claude.
4. Instala y corre la calculadora.
5. Añade un campo.
6. Guarda los datos en CSV.
7. Despliega un v1.
8. Añade un login simple.
9. Itera con Claude Code.
10. Define el alcance del v1.



Recursos

1. Streamlit.
2. Claude Code.
3. Flask / FastAPI.
4. Plantilla de app.
5. Documento de la API.
6. Guía de despliegue.
7. SQLite.

8. Ejemplos de apps internas.
9. Checklist de producto.
10. Cookbook.

20. automatizaciones-empresa · 8 automatizaciones empresariales

Casos extremo a extremo que conectan las 4 áreas. Etiquetas: BOM → factura, OC automática, Pipeline CRM, Cierre mensual.

Introducción

1. Cierre del programa.
2. Automatizaciones de extremo a extremo.
3. Conectan ingeniería, compras, CRM y contabilidad.
4. Implementa una, mídela y encadena la siguiente.
5. Quién: todo el estudio.
6. Mitos sobre automatizar la empresa.
7. Una hoja de ruta.
8. Aplicación directa.
9. Riesgos: encadenar sin control.
10. Escalar con datos.

Fundamentos (las 8 + criterio)

1. Factura → asiento contable.
2. BOM → orden de compra.
3. Email de cliente → oportunidad en CRM.
4. Proyecto técnico → propuesta comercial.
5. Pipeline CRM con seguimiento.
6. Conciliación bancaria asistida.
7. Cierre mensual semiautomático.
8. Informe de dirección automático.
9. Orden de implementación (esfuerzo vs. impacto).
10. Humano en las decisiones críticas.

Ejemplos prácticos

1. Flujo factura por email (5 pasos).
2. BOM → pedido optimizado.
3. Email → deal en CRM.
4. Coste → oferta.
5. Aviso de oportunidad fría.
6. Cruce extracto-factura.
7. Checklist de cierre mensual.

8. Informe mensual a dirección.
9. Encadenar dos módulos del curso.
10. Punto de aprobación humana.

Ejercicios (~2,5 min)

1. Elige la automatización de más impacto.
2. Dibuja su flujo en 4-5 pasos.
3. Asigna un módulo del curso a cada paso.
4. Marca los puntos humanos.
5. Ordena las 8 por esfuerzo vs. impacto.
6. Define tu proyecto piloto.
7. Esboza disparador y salida.
8. Estima el ahorro.
9. Define las métricas de éxito.
10. Plan de escalado.

Recursos

1. Building effective agents.
2. Use case guides.
3. Cookbook.
4. Plantillas de flujo.
5. Documento de tool use / MCP.
6. Casos de empresa.
7. Checklist de seguridad.
8. Guía de ROI.
9. Ejemplos de orquestación.
10. Plantilla de roadmap.